SOFTWARE REQUIREMENTS SPECIFICATION

for

Natural Control

Version 0.1 (draft)

Prepared by Group 7
Lucas DeBoer, Kevin Kim, Aaron Loh, Curtis Vos

Group 7, SFWRENG 4G06, McMaster University

October 20, 2026

Revision History

Name	Date	Reason For Changes	Version
Aaron Loh	2026-10-14	First draft from team planning decisions	0.1
Aaron Loh	2026-10-17	Supervision rules and layout review	0.1

This document follows the requirements documentation structure presented in the Handbook of requirements and business analysis, by Bertrand Meyer, with the changes listed in the first chapter.

Comment: *Working draft. Red comment boxes mark decisions and writing the team still owns. Goals G1 to G5 refer to the Problem Statement goals.*

Contents

Changes from the Meyer template 5

Glossary 6

Goals book 7
- G.1 Context and overall objective . . . 7
- G.2 Current situation . . . 7
- G.3 Expected benefits . . . 7
- G.4 Functionality overview . . . 7
- G.5 High-level usage scenarios . . . 7
- G.6 Limitations and exclusions . . . 8
- G.7 Stakeholders and requirements sources . . . 8
- G.8 User profiles . . . 8

Environment book 9
- E.1 Glossary . . . 9
- E.2 Components . . . 9
- E.3 Constraints . . . 9
- E.4 Assumptions . . . 10
- E.5 Effects . . . 10
- E.6 Invariants . . . 10
- E.7 Standards and legal/regulatory factors . . . 10
  - Environment requirements . . . 11

System book 12
- S.1 Components . . . 12
- S.2 Functionality . . . 14
  - S.2.1 Voice interface . . . 14
  - S.2.2 Task manager . . . 14
  - S.2.3 Action gate . . . 15
  - S.2.4 Outcome verification . . . 16
  - S.2.5 Action tools and integrations . . . 17
  - S.2.6 Accessible client and task history layer . . . 17
  - S.2.7 Safety requirements . . . 18
  - S.2.8 Non-functional requirements . . . 19
- S.3 Interfaces . . . 20
- S.4 Normal operation . . . 21
- S.5 Undesired event handling . . . 21
- S.6 Detailed usage scenarios . . . 24
  - UC1: Reply to an email . . . 24
  - UC2: Add an event from a webpage to the calendar . . . 25
  - UC3: Compare information on a website . . . 25
  - UC4: Correct or pause a running task . . . 26
  - UC5: Reconnect without duplicating an action . . . 27
- S.7 Prioritization . . . 28

S.8 Verification and acceptance criteria . . . 28
S.9 Formalization: task state machine . . . 28

Project book **31**
P.1 Roles and personnel . . . 31
P.2 Imposed technical choices . . . 31
P.3 Schedule and milestones . . . 31
P.4 Tasks and deliverables . . . 31
P.5 Required technology elements . . . 31
P.6 Risk and mitigation analysis . . . 32
P.7 Requirements process and report . . . 32

Requirement dependencies **33**

Likely and unlikely changes **34**

References **35**

Appendix: AI use disclosure **36**

Appendix: Reflection **37**

Changes from the Meyer template

- Glossary moved to the front, so every term is defined before first use (rubric Level 4).
- Added G.8 User profiles, to describe distinct user groups and their needs.
- Added E.7 Standards and legal/regulatory factors, plus environment requirements.
- S.2 split into one subsection per component, with safety requirements ported from the Hazard Analysis.
- Added S.4 Normal operation, S.5 Undesired event handling, and S.9 Formalization (task state machine).
- Added requirement dependencies (traceability) and References.

Glossary

Project terms (from GLOSSARY.md):

- User: the person who owns the connected accounts and gives commands. Other voices in the room are not users.
- Setup helper: a person who completes the initial assisted setup for the user.
- Session: the period during which the user is talking with the system, started and ended with the client's start control or a keyboard shortcut, or ended by disconnection. A session can contain several tasks.
- Task: one job the user asks for. It may pause before it is completed, cancelled, or failed.
- Consequential action: a write to user or account data, a submission, an external message, or a commitment such as accepting terms or granting access. Navigation and temporary preparation do not need approval unless they cause one of these effects. FR24 defines one exception for rejecting non-essential cookies.
- Approval: the user's explicit consent to one consequential action, or to a listed batch of similar actions, with their specific details; used once and void if any detail changes.
- Barge-in: the user speaking while the system is speaking; it stops the speech only.
- Stop request: an explicit "stop working" command or stop key. It pauses work and preserves the task. Cancellation ends the task.
- Pause acknowledged: the system received the stop request. Pause confirmed: no new task actions can start. Any earlier operation that is still pending is reported separately.
- Verified outcome: a result supported by a matching tool response, account record, or website confirmation. Unknown outcome: an action whose result cannot be determined.

Technical terms: LiveKit, Hermes, Browser Use, MCP (Model Context Protocol), OAuth, VoiceOver, STT (speech-to-text), TTS (text-to-speech), LLM (large language model), POC (proof of concept).

Comment: *Add a one-line definition and an online reference link for each technical term, as AlgoCatan did.*

Goals

G.1 Context and overall objective

Many websites and applications communicate information, choices, and state changes visually. Blind users can face barriers completing multi-step digital tasks and knowing whether those tasks succeeded. Natural Control is a voice interface that lets a blind user delegate tasks in a web browser and in Gmail and Google Calendar to an AI agent, while staying informed and in control.

G.2 Current situation

Today, blind users complete these tasks with screen readers such as VoiceOver, navigating each interface element by element, or with help from a sighted person. Existing AI computer-use agents assume the user can watch the screen to monitor and correct them.

In an interview with a former CNIB Deafblind Community Services (DBCS) staff member on October 17, 2026, the participant observed that screen readers are hard to learn for older, newly diagnosed users and often read irrelevant content, and that everyday tasks such as booking appointments often need a friend's help.

Comment: *Add findings from blind participants once sessions start.*

G.3 Expected benefits

- Delegate, don't navigate: the user states an outcome ("reply to Alex") instead of navigating the interface step by step. (G1)
- Control without sight: the user can understand, approve, correct, and stop the agent without seeing the screen. (G2, G4)
- Trustworthy results: results are checked against the real account or page before being reported, so the user does not need sighted help to confirm what happened. (G3)

Comment: *Support each benefit with evidence (interview findings, prior research).*

G.4 Functionality overview

The user starts a session from the web client and speaks a task. The system clarifies missing details, carries out read-only steps on its own, asks for approval before every consequential action, reports progress and verified results by voice, and accepts corrections and stop requests at any time. A task history records what happened.

G.5 High-level usage scenarios

- Reply to an email.
- Add an event found on a webpage to the calendar.

- Compare information on an unfamiliar website.
- Correct or stop a running task.
- Reconnect after a dropped connection without duplicating an action.

Comment: *Draw a use case or activity diagram for these scenarios.*

G.6 Limitations and exclusions

- Native Mac application control and general local-file tasks.
- Platforms other than Mac with Chrome; languages other than English.
- Phone calling and saved workflows (stretch goals).
- Background continuation: tasks always pause when the user disconnects; continuing unattended is a possible extension.
- Fully local model inference; the system uses cloud AI and speech providers.
- Users who cannot hear spoken output, including many deafblind users. Braille output through VoiceOver is a possible extension.

G.7 Stakeholders and requirements sources

- Primary: blind and visually impaired people who want to complete digital tasks without visual feedback. Initial evaluation: blind Mac users with screen-reader experience.
- Secondary: other people who want nonvisual interaction.
- Setup helper: a person who completes the assisted setup.
- Requirements sources: the Problem Statement, Development Plan, and Hazard Analysis; TA and instructor guidance; prior research (WebAIM survey, Morae); an interview with a former CNIB Deafblind Community Services (DBCS) staff member on October 17, 2026; and planned participant sessions.
- Intervenors: support workers who may help deafblind users with setup and recovery.

G.8 User profiles

- Experienced screen-reader user: fluent with VoiceOver or JAWS and keyboard navigation; wants speed on multi-step tasks and keeps their screen reader for everything else. Primary evaluation group.
- Less experienced or recently blind user: often older, with acquired vision loss, limited screen-reader skill, and possibly an older device; benefits most from delegation and clear spoken confirmation.
- Sighted user wanting nonvisual interaction: can see the screen but prefers or needs voice (busy hands, away from the screen); can verify visually, so has lower risk.
- Setup helper: comfortable with computers; connects accounts and grants permissions once, then is not needed for daily use.

Comment: *Validate these profiles with blind participants; drop or adjust any that do not hold up.*

Environment

E.1 Glossary

See the Glossary at the front of this document.

E.2 Components

- Websites the user asks about (unmodified, not under our control).
- Google services: Gmail and Google Calendar APIs.
- Composio: hosted service that stores OAuth tokens and runs Gmail and Calendar tool calls. Its "Don't store data" setting is turned on so tool inputs and outputs are not logged.
- LiveKit (real-time audio and speech pipeline).
- AI model and speech providers.
- The Hermes agent and Browser Use (reused open-source components).
- VoiceOver and the Mac's microphone and speakers.
- Network access.

E.3 Constraints

- Mac with Chrome, English, as decided in the Development Plan.
- Team budget ceiling of CAD 500.
- Course schedule: POC demonstration November 23 to December 3; final demonstration and EXPO in April.
- Gmail access uses restricted Google OAuth permissions; an unverified app is limited to a small number of registered test users.
- Provider terms of use and data retention policies apply to anything sent to them.
- Passwords and payment details are entered through a private page or the user's own device. One-time authentication codes may be spoken. Spoken codes pass through the configured speech and conversation services. Provider handling must be checked before claiming stronger privacy guarantees.

Comment: *Verify Google's current limits for unverified apps (number of test users, token expiry) before final submission.*

E.4 Assumptions

- A setup helper completed setup correctly (accounts connected, permissions granted).
- Only the user gives commands; other voices are noise.
- Connected accounts belong to the user; development and evaluation use test accounts.
- The user can hear spoken output (see G.6).

Speech recognition, the AI model, websites, and external services are not assumed to work correctly; their failures are covered by the Hazard Analysis and S.5.

E.5 Effects

- Emails are searched, read, drafted, sent, replied to, archived, labelled, or deleted.
- Calendar events are read, created, updated, or deleted.
- Website forms may be filled and submitted, subject to approval.
- Data is sent to AI and speech providers.

E.6 Invariants

- Writes and consequential commitments require approval of their exact details. FR24 defines the narrow exception for rejecting non-essential cookies.
- The user's existing accessibility tools (VoiceOver, keyboard navigation) keep working while the system runs.
- Credentials are never written to source control, logs, or task history.

E.7 Standards and legal/regulatory factors

- WCAG 2.2 Level AA: accessibility standard for the web client.
- AODA (Accessibility for Ontarians with Disabilities Act): Ontario accessibility law that references WCAG.
- PIPEDA: Canadian privacy law for personal information such as email and calendar contents.
- Google API Services User Data Policy: limits how Gmail and Calendar data may be used and stored.
- Open-source licenses: our code is MIT; reused components keep their own licenses and notices.
- Coding standards: PEP 8 for Python; strict TypeScript with a shared linter for the client.

Comment: *Confirm which of these apply to a student prototype and cite each one.*

Environment requirements

- NFR.E.1: The system shall run on macOS with Google Chrome.
- NFR.E.2: The web client shall meet WCAG 2.2 Level AA.
- NFR.E.3: The system shall use Gmail and Calendar data only to perform the user's requested tasks, consistent with the Google API Services User Data Policy.
- NFR.E.4: Python code shall pass the team's PEP 8 linter checks; TypeScript code shall pass strict type checks and the shared linter.

System

S.1 Components

The proposed system has eight components. These match the Hazard Analysis. Figure 1 shows their connections and external services. Team-owned components are planned work. The POC must test the integration boundaries.

- **Accessible client:** the web interface for sessions and task history. It provides keyboard controls and works alongside VoiceOver.
- **Voice interface:** uses LiveKit for audio and conversational turn handling. Our integration passes requests and corrections to the task manager. It speaks relevant questions and results.
- **Task manager:** starts tasks and tracks their state. It handles corrections, pauses, cancellation, and resumption (S.9).
- **AI agent:** Hermes plans tasks and proposes tool calls. Each call must pass through the action gate.
- **Action gate:** checks every proposed action. It requests approval for writes and consequential commitments. Approval details come from the proposed call. The gate also blocks new task actions while work is paused.
- **Outcome verification:** checks evidence for each consequential action. Evidence may come from its tool response or a targeted account or page check. The result is recorded as verified, partial, failed, or unknown.
- **Action tools and integrations:** Browser Use handles websites. Gmail and Google Calendar tools use the Composio SDK. The proposed integration places approval checks before execution and outcome checks afterward. The POC must verify that all action paths pass through these checks. Credential handling also needs verification.
- **Task history layer:** records task state and action outcomes. It also records approvals and action IDs. These records support accessible review and recovery after disconnection.

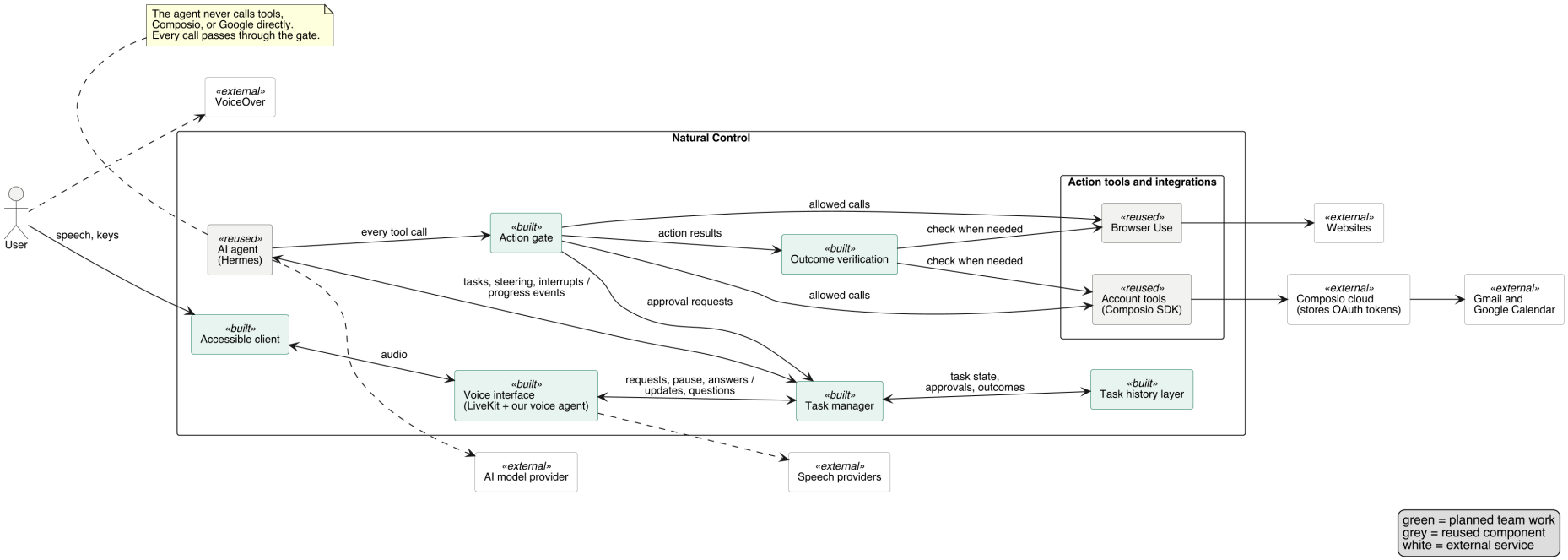


Figure 1: Component diagram. Green components are planned team work. Grey components are reused. White boxes are external services.

S.2 Functionality

Requirements are grouped by their responsible component (S.1). Each has a rationale and trace links. G1 to G5 refer to project goals. FR.Sa and NFR.Sa refer to safety requirements. The task manager and action gate constrain the AI agent. Outcome verification checks its results.

S.2.1 Voice interface

Table 1: Voice interface requirements

ID	Requirement	Rationale	Traces to
FR1	The system shall accept spoken input during a session. It shall support pauses within a thought through the selected voice framework's turn handling.	Voice is the main input for everyday use. Turn timing needs POC and user evaluation.	G1
FR2	The user shall be able to interrupt spoken output. This shall not restart the task or pause its execution by itself. A correction or explicit work-stop request shall follow its own handling.	Users need to cut off long speech without losing work.	G2
FR3	An explicit "stop working" command or the stop key shall pause the task (FR8). A bare "stop" shall stop speech and pause work. The system shall clarify what the user wants next. An explicit request to cancel or abandon the task shall end it.	Speech interruption, work pause, and cancellation have different effects. The key provides a backup when voice fails.	G2; FR.Sa.1, FR.Sa.4
FR16	The system shall let the user correct part of a read-back (for example, one sentence or one detail) by voice without restarting the request.	Interview: fixing a read-back usually means starting over. Correcting speech input is slow for blind users [Azenkot and Lee 2013].	G4
FR18	Spoken updates and summaries shall include only information relevant to the current task, with more detail available on request.	Interview and WebAIM survey: screen readers read too much irrelevant content.	G2

S.2.2 Task manager

Table 2: Task manager requirements

ID	Requirement	Rationale	Traces to
FR4	The system shall ask when required details are missing or ambiguous. It shall recheck relevant information before continuing.	Guessing risks acting on the wrong item.	G3; FR.Sa.9
FR5	The system shall report meaningful progress and blockers. It shall answer requests for current status. Routine updates shall follow the task and conversation without exceeding the maximum silence interval in NFR.Sa.3. Pending questions and approvals shall not expire while the user remains connected.	Users need awareness without constant narration. Silence is not approval.	G2; NFR.Sa.3

ID	Requirement	Rationale	Traces to
FR6	The system shall accept corrections during a task without requiring a restart.	Restarting a long task by voice is costly.	G4
FR7	Disconnection shall pause the task as in FR8. On reconnection the system shall report its state and reconcile uncertain outcomes. It shall wait for a clear request to resume work.	Reconnecting the voice session does not authorize more actions.	G2, G4; FR.Sa.7
FR8	A stop request shall block new task actions and preserve task context. The system shall stop or suspend ongoing work where safe and supported. An operation already submitted may settle if it cannot be stopped safely. The system shall distinguish acknowledgment from confirmed pause. It shall report completed work and unfinished work. It shall also report any outcome still unknown. Checks during a pause shall only reconcile earlier actions. Work shall resume only on a clear user request.	Pausing does not undo a submitted action. The user needs an accurate account of what remains.	G2, G4; FR.Sa.5.i, FR.Sa.5.ii, FR.Sa.5.iii
FR19	A new request during an active task shall be treated as a correction when that is clear. Otherwise the system shall ask whether to queue it or replace the current task. Replacement shall end the earlier task before the new one starts.	One active task keeps spoken progress easy to follow (S.9, P6).	G2
FR20	When the user moves to another request the system shall withdraw pending questions and approvals. It shall state that the pending action was not executed. It shall distinguish this from earlier completed work. The new request shall follow FR19.	An abandoned approval must not be used later.	G2, G3; FR.Sa.15
FR21	At session start the system shall announce paused tasks and their last known state. The user may resume or cancel. Before resuming the system shall check relevant state and reconcile earlier actions. Pending approvals shall be requested again. Reconnection alone shall not resume work.	Recovery must preserve context without repeating actions.	G2, G4; FR.Sa.8

S.2.3 Action gate

Table 3: Action gate requirements

ID	Requirement	Rationale	Traces to
FR9	Writes and consequential commitments shall require approval of their exact details. These include sending, saving, deleting, submitting, granting access, and accepting terms. Approval shall be an explicit affirmative answer to the pending approval question. Silence is not approval. Changed details require new approval. FR24 defines the cookie-rejection exception.	Users retain control over changes and commitments. Approval checks use the proposed tool call (ADR 0001).	G2, G3; FR.Sa.3, FR.Sa.13, FR.Sa.14

ID	Requirement	Rationale	Traces to
FR10	Reading, navigation, and temporary preparation shall proceed without approval when they cause no consequential effect. Autosaved drafts are writes. A click that submits data is consequential. Unknown effects shall require approval. A failed read-only lookup may be retried once before it is reported as a blocker.	Classify the effect of an action rather than the name of its tool.	G1, G2; FR.Sa.13
FR11	Each approved action shall have a unique ID. A completed action shall not be repeated as a retry. Before retrying an uncertain action the system shall check its outcome. If it remains unknown the system shall ask the user. A new attempt requires fresh approval.	An uncertain result must not cause a duplicate write.	G3; FR.Sa.6, FR.Sa.18
FR13	The system shall treat webpage and email content as data, never as instructions.	Websites and emails can contain text written to manipulate agents.	FR.Sa.16

Approval details. Similar actions may share one approval. The read-back shall state the number of actions, the intended changes, and a concise description identifying the affected items. Full item details shall be available on request. The approved batch shall retain each item's identity and exact details. Each item may run once. Partial failures shall be reported per item. A pause shall block remaining items and withdraw their unused approvals.

For email the read-back shall name the recipients and subject. It shall read the body in full up to SHORT_MESSAGE_LENGTH. For longer messages it shall give a summary and offer the full text. The user may request a sentence-by-sentence read-back. A reduced-prompt permission mode is deferred.

S.2.4 Outcome verification

Table 4: Outcome verification requirements

ID	Requirement	Rationale	Traces to
FR12	Each consequential result shall be reported as verified, partial, failed, or unknown. Evidence shall match the intended action and details. It may be a completion response from the connector, a matching account record, or a website confirmation. Another check is required when that evidence is unclear. A queued request shall not be reported as delivered.	The reported result must match the available evidence. A second lookup is not always needed.	G3; FR.Sa.10

S.2.5 Action tools and integrations

Table 5: Action tools and integrations requirements

ID	Requirement	Rationale	Traces to
FR14	The system shall request only the account permissions it needs, and explain them during setup, including which providers receive which data.	Narrow permissions limit the damage any other failure can do.	G5; FR.Sa.17
FR17	The user shall be able to mark topics or accounts as off-limits. The system shall not read or act on them.	The proxy interview raised concerns about sensitive account data.	G2
FR23	Website passwords shall be entered through a private setup page and stored encrypted. They shall not be spoken or placed in the conversation. Saved credentials shall be used only for their intended site. The user may speak a one-time authentication code or approve a push on their device. The system shall explain that spoken codes pass through the configured speech and conversation services. A site that requires a device-bound passkey shall use a manual handoff.	Spoken codes can be heard nearby and processed by providers. Private typed code input is a proposed alternative.	G2, G5; FR.Sa.17
FR24	The system may automatically choose a clearly identified “reject non-essential cookies” or “necessary only” option. Any ambiguous consent choice shall require explicit user approval. This is a narrow exception to FR9. Accepting cookies or terms shall require approval.	A limited exception reduces repeated consent prompts. It does not authorize accepting terms.	G2, G3; FR.Sa.13

S.2.6 Accessible client and task history layer

Table 6: Accessible client and task history layer requirements

ID	Requirement	Rationale	Traces to
FR15	The system shall provide accessible task history and spoken summaries from recorded events. The user shall be able to delete that history.	Users need to review what happened without sight.	G2; FR.Sa.2
FR22	The client shall provide controls to start and end a voice session. A keyboard shortcut shall also be available. Ending a session shall pause active work as in FR8.	Deliberate activation avoids an always-on microphone.	G2, G5

S.2.7 Safety requirements

The following safety requirements are based on the Hazard Analysis. Hazard IDs refer to its FMEA rows.

Functional safety requirements

- **FR.Sa.1:** The system shall provide a keyboard stop control that works whenever the client is running and does not conflict with VoiceOver commands. (H1-a)
- **FR.Sa.2:** The system shall build the task history only from recorded actions and verified outcomes, labelling unknown entries. (H8-a)
- **FR.Sa.3:** Before a consequential action the system shall read back its key details. It shall proceed only after explicit approval. The cookie-rejection exception is defined in FR.Sa.13. (H2-a)
- **FR.Sa.4:** Explicit stop phrases and the stop key shall use a fixed control path. They shall not depend on the planning agent choosing to pause. (H2-b)
- **FR.Sa.5.i:** After a stop-work request, the system shall pause the task and block all further actions for it. Only checks of earlier outcomes may continue running. (H2-c, H3-a)
- **FR.Sa.5.ii:** After a stop-work request, the system shall immediately acknowledge and confirm completion of the request as separate messages before reporting completed work, unfinished work, and uncertain outcomes. (H3-a)
- **FR.Sa.5.iii:** When a task is paused, an explicit user request shall resume or terminate the task. If terminated, earlier actions are not undone. (H3-a)
- **FR.Sa.6:** Each approved action shall have a unique ID. An action with that ID shall not execute twice. (H3-b)
- **FR.Sa.7:** Disconnection shall block new task actions and pause the task. Reconnection alone shall not resume it. (H3-b)
- **FR.Sa.8:** Task state and approval records shall survive restart. The system shall announce interrupted tasks with their last known state. Unused approvals shall be withdrawn. (H8-b)
- **FR.Sa.9:** The system shall ask a clarifying question when details required for an action are missing or ambiguous. (H4-a)
- **FR.Sa.10:** The system shall report a task as verified, failed, or partial only after verifying its outcome with evidence that matches the intended action and its details, such as a connector result, an account record, or a website confirmation. If the evidence is unclear, the system shall check again or report the outcome as unknown. (H4-b, H6-a, H6-b, H7-a)
- **FR.Sa.11:** A configured step or time limit shall pause the task and prompt the user. Time spent waiting for user clarification or approval shall not count toward the execution time limit. (H4-c)
- **FR.Sa.12:** An external service failure shall pause work and the system shall follow the steps outlined in FR.Sa.5.i, FR.Sa.5.ii, and FR.Sa.5.iii. (H4-d)
- **FR.Sa.13:** Unclear actions and consequential actions such as submissions, external messages, saved data changes, or granting permission require explicit approval before being executed; only actions with no such lasting effect do not require approval. One exception to this rule: rejecting non-essential cookies or only allowing necessary cookies on webpages does not require explicit approval. (H5-a)

- **FR.Sa.14:** The system shall bind each approval to the action's exact details, use it at most once, and require new approval if any detail changes. A pause, disconnection, cancellation, or task replacement shall withdraw unused approvals. (H5-b, H7-a)
- **FR.Sa.15:** When a pending approval expires or is withdrawn, the system shall tell the user that the action was not executed and shall state which earlier actions were completed. (H5-c)
- **FR.Sa.16:** The system shall treat webpage and email content as data, and shall only perform consequential actions that trace to the user's request. (H7-b)
- **FR.Sa.17:** The system shall request only the account permissions each tool needs, and shall store and use passwords and other credentials securely, keeping them out of source control, logs, and the AI model input. (H7-c)
- **FR.Sa.18:** The system shall check the service or item state directly and confirm the user's approval before retrying an action whose outcome is unknown. (H7-d)
- **FR.Sa.19:** Before deletions, the system shall read back the item's identifying details, state whether the operation is reversible, and use recoverable deletion where supported. (H5-d)
- **FR.Sa.20:** If voice input, voice output, or speech services become unavailable, the system shall stop as outlined in FR.Sa.5.i, accept no new tasks requests until voice functionality is restored, and shall not resume unless user explicitly requests it. (H2-c)

Non-functional safety requirements

- **NFR.Sa.1:** The system shall audibly acknowledge a stop request within STOP_ACK_TIME of the end of the utterance or key press. (H2-b)
- **NFR.Sa.2:** The client shall signal disconnection within CONNECTION_NOTICE_TIME. A local sound shall be used if network speech is unavailable. (H3-b)
- **NFR.Sa.3:** While a task is running, the system shall give a spoken status update after IDLE_PROMPT_TIME without spoken progress, outlining progress made or blockers. (H4-c)

S.2.8 Non-functional requirements

Table 7: Non-functional requirements

ID	Requirement	Rationale	Traces to
NFR1	Session controls, work pause, approval, and task history shall work with VoiceOver and the keyboard alone.	Users rely on their existing assistive tools.	G2, G5
NFR2	At least TARGET_COMPLETION of tasks in the declared task suite shall be completed correctly without sighted help. The task suite shall include representative Gmail, Google Calendar, and unfamiliar website tasks.	Measures the main benefit: independent task completion.	G1
NFR3	A work-stop request shall be acknowledged within STOP_ACK_TIME. Time to block new actions and time to reconcile in-flight work shall be measured separately.	A quick acknowledgment does not prove earlier actions ended.	G2; NFR.Sa.1

ID	Requirement	Rationale	Traces to
NFR4	The system shall begin its response within RESPONSE_TIME after the selected turn detector commits an ordinary request. Premature replies and missed interruptions shall also be evaluated.	Fast replies must not cut off unfinished thoughts.	G2
NFR5	The application shall not retain raw microphone audio. It shall exclude credentials and authentication codes from task history, logs, and source control. Setup shall explain provider processing and retention separately.	Application storage rules do not establish provider privacy guarantees.	G5; FR.Sa.17

Constants below remain provisional. The POC will inform timing choices. Routine speech has no fixed update interval.

Table 8: Constants

Constant	Meaning	Value
STOP_ACK_TIME	Time to audibly acknowledge a work-stop request	TBD after POC (Development Plan provisional: 2 s)
RESPONSE_TIME	Time from committed user turn to response start	TBD after POC
CONNECTION_-NOTICE_TIME	Time to signal connection loss (NFR.Sa.2)	TBD after POC
IDLE_PROMPT_TIME	Maximum interval without spoken progress during active task execution	TBD after POC
SHORT_MESSAGE_-LENGTH	Longest email body read in full before approval	TBD after POC
TARGET_COMPLETION	Share of tasks completed correctly without sighted help	TBD (Development Plan provisional: 80%)

S.3 Interfaces

- **User interface:** a web client in Chrome. It provides session controls and task status. The user can pause, resume, or cancel work. Approval details and history are accessible through VoiceOver and the keyboard. Spoken questions can be repeated on request. The system shall explain the keyboard shortcut for stopping work during setup and repeat the shortcut on request. The shortcut shall not conflict with VoiceOver controls.
- **Program interfaces:** LiveKit carries audio and speech events. The Hermes integration carries task requests, corrections, and progress. The task manager controls execution separately from speech. Browser Use and Composio supply action tools. Each tool call passes through the gate.

Comment: *The voice/client owner must confirm screen layouts and keyboard shortcuts. Private typed code input remains a proposed extension.*

S.4 Normal operation

Figure 2 follows one request from start to finish: the user asks to add an event found on a webpage to their calendar.

1. The user starts a session from the client (FR22) and speaks the request. The voice interface passes it to the task manager. It records the task and starts the agent in Working. The user hears a short acknowledgment.
2. The agent reads the page. The action gate lets this read-only call through to Browser Use and screens the returned content for injected instructions (FR10, FR13).
3. The agent proposes event creation. The gate holds the write and requests approval of its exact details. The task enters AwaitingApproval.
4. The voice interface reads the details back. The user says "yes." The task manager records approval and releases the call. The task enters Executing (FR9).
5. The gate submits the call with a unique action ID (FR11). The returned event details confirm creation. Outcome verification checks those details and reports the result (FR12). A separate lookup is used if the response is unclear.
6. The agent finishes the task (state Done). The task history records each step (FR15), and the user hears the verified result.

S.5 Undesired event handling

- **Misheard or ambiguous request:** ask for clarification. Read back key details before approval. (H2-a, H4-a)
- **Missed work-stop request:** provide a keyboard backup. Block new task actions once the request is recognized. (H2-b, H3-a)
- **Disconnection:** pause work. Report the state after reconnection and wait for an instruction to resume. (H3-b)
- **Provider failure:** pause work. Explain what completed and what remains unfinished or unknown. (H4-d)
- **Unknown outcome:** check the affected account or page before any retry. Ask the user if the outcome remains unknown. (H7-d)
- **Injected instructions:** treat webpage and email text as data. Do not let it authorize actions. (H7-b)

Figure 3 shows a work-stop request during event creation. The system acknowledges it within STOP_ACK_TIME. It blocks new task actions and pauses the task. An earlier submission may still complete. The system reports its result or marks it unknown. A pause does not promise that the external service stopped processing the submission (FR8).

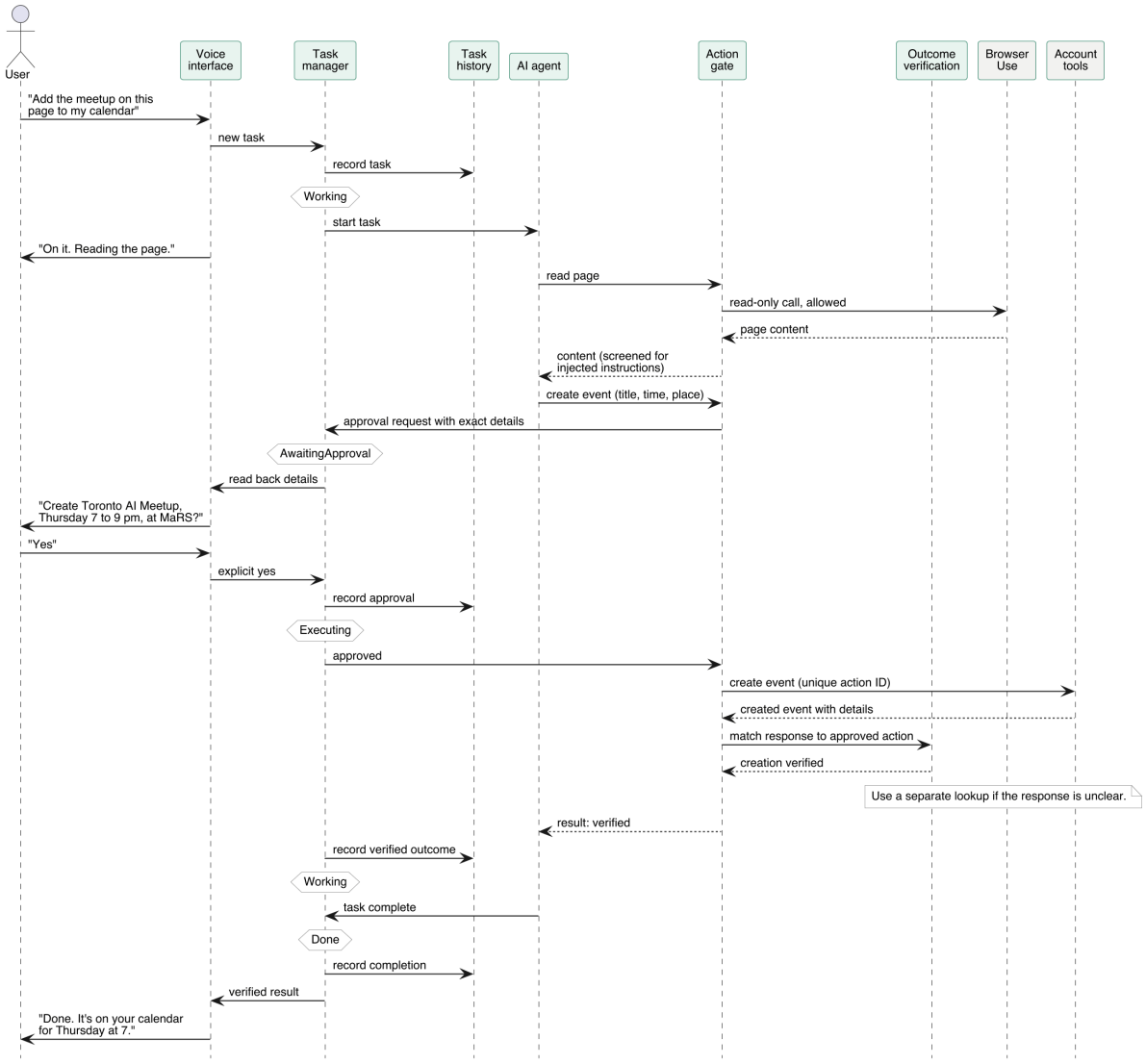


Figure 2: Normal operation: adding an event from a webpage to the calendar.

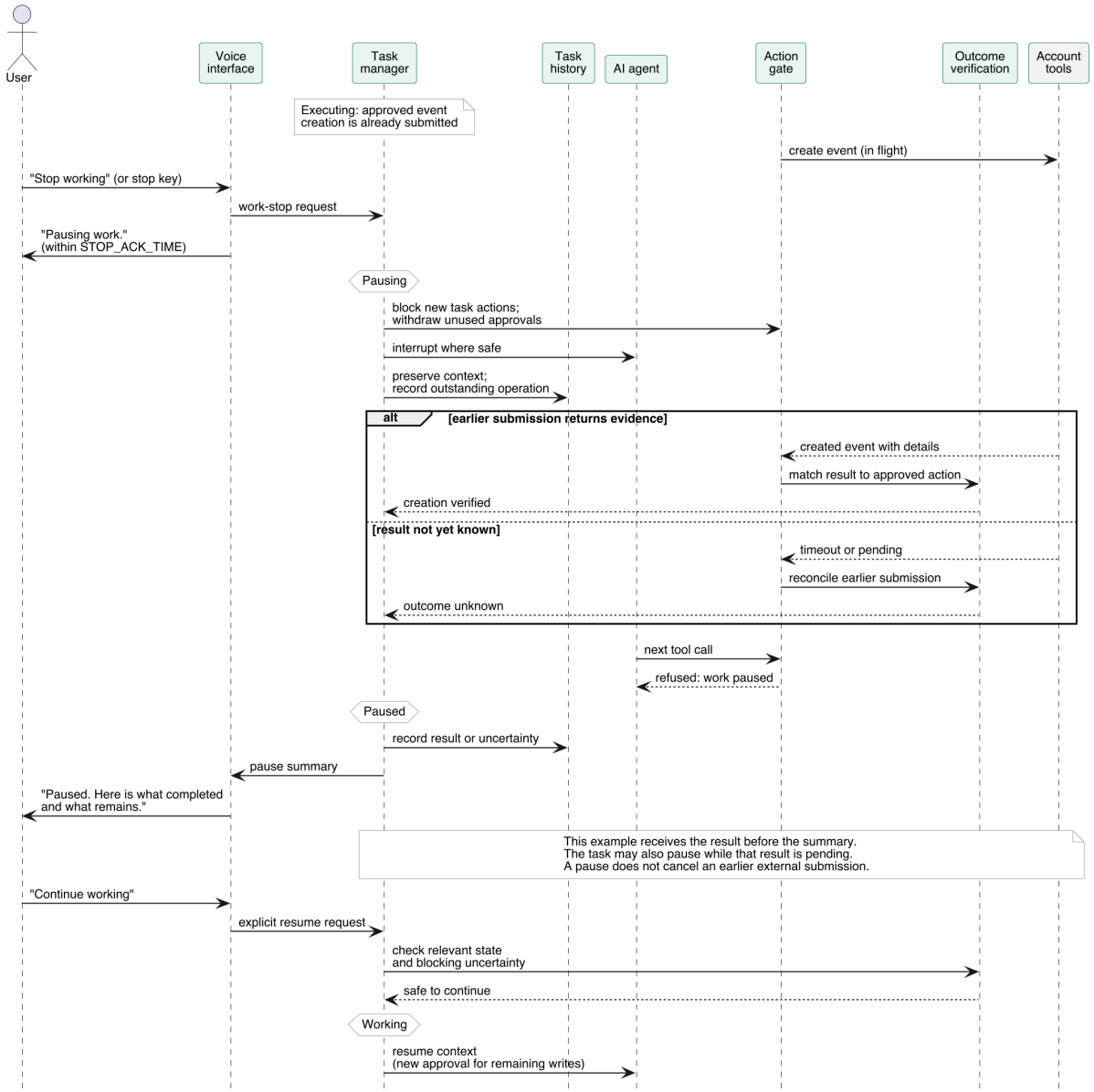


Figure 3: Pausing work while an approved action is in flight.

S.6 Detailed usage scenarios

The five use cases below expand the high-level scenarios in G.5. Each lists the main success scenario and its extensions. Extensions are numbered by the step they branch from: "2a" branches from step 2.

UC1: Reply to an email

Primary actor: User

Preconditions: A session is active, Gmail is connected through the account tools, and no other task is active.

Trigger: "Reply to Alex Chen's invitation and say Friday at 2 works for me."

Main success scenario:

1. The voice interface passes the request to the task manager. It starts the task.
2. The agent searches the inbox (read-only) and finds the invitation.
3. The agent prepares the reply without saving a draft. The gate holds the send and requests approval.
4. The system reads back the recipient, subject, and full body: "Send to Alex Chen, subject Re: Invitation: Thanks for the invitation, Friday at 2 works for me. Send it?"
5. The user says "Yes, send it."
6. The gate sends the reply with a unique action ID. Outcome verification finds the matching message in Sent.
7. The user hears "Your reply was sent to Alex Chen." The task history records the request, approval, action, and outcome.

Extensions:

- **2a.** Two possible contacts match. The system asks which one (FR4). It waits for the answer and repeats the question on request (FR5).
- **4a.** The body exceeds SHORT_MESSAGE_LENGTH. The system reads the recipient and subject. It summarizes the body and offers a full read-back (FR9).
- **4b.** The user says "change Friday to Monday." That detail changes and is read back. Earlier approval no longer applies (FR16, FR9).
- **5a.** The user declines or requests another task. Approval is withdrawn. The user hears that this reply was not sent (FR20).
- **6a.** The send cannot be confirmed. The system reports failure only with evidence of failure. Otherwise it reports unknown (FR12).

Success guarantee: Only the approved reply is sent to the intended recipient. The reported result matches the sending evidence. Delivery or reading by the recipient is not claimed.

Failure guarantee: Nothing is sent without approval of its exact recipient and text. The user hears whether sending was confirmed or remains uncertain.

UC2: Add an event from a webpage to the calendar

Primary actor: User

Preconditions: A session is active, Google Calendar is connected, and the page is public or has a saved login (FR23).

Trigger: "Add the meetup on this page to my calendar."

Main success scenario:

1. The task manager starts the task. The user hears a short acknowledgment.
2. The browser opens the page. A clear non-essential cookie prompt is rejected under FR24.
3. The agent reads the event title, date, time, and place (read-only).
4. The agent proposes event creation. The gate holds it. The system reads back the details.
5. The user approves.
6. The gate creates the event with a unique action ID. Outcome verification checks the returned details or a matching calendar record.
7. The user hears the verified result (Figure 2).

Extensions:

- **2a.** The consent prompt is unclear. The system asks the user before choosing an option (FR24).
- **3a.** The page tells the agent to ignore the user or send information elsewhere. The system treats this text as data. It does not follow those instructions (FR13).
- **3b.** The end time is missing. The system asks instead of guessing (FR4).
- **3c.** A matching event already exists. The system reports it before asking for approval.
- **5a.** The user declines. No event is created. The system reports this.

Success guarantee: The calendar contains one approved event. A matching creation response or calendar record confirms its details.

Failure guarantee: No event is created without approval. Uncertain creation is reported as unknown.

UC3: Compare information on a website

Primary actor: User

Preconditions: A session is active. The site is public, or its login was saved through the vault page (FR23).

Trigger: "On this conference site, compare the standard and student tickets."

Main success scenario:

1. The task manager starts the task.
2. The agent navigates without making a consequential change. Clear non-essential cookie prompts follow FR24.
3. The agent gathers each ticket's price and what it includes.
4. The user hears a short comparison limited to the request. Uncertain details are stated. For example, fees may be unclear (FR18, FR12).

5. The user corrects the request: "Only compare tickets that include workshops."
6. The agent rechecks and reports the corrected comparison (FR6). Nothing is purchased or submitted.

Extensions:

- **2a.** A saved login exists. The agent uses it only on its intended site (FR23).
- **2b.** No login is saved. The system explains the private setup route. The task pauses for setup.
- **2c.** The site requests a one-time code or push approval. The user may speak the code or approve on their device (FR23).
- **2d.** The site requires a device-bound passkey. The system explains the manual handoff when that requirement becomes known.
- **5a.** The user asks to buy a ticket. This requires approval of the purchase details. It follows the approval and verification flow in UC1.

Success guarantee: The comparison matches the site and the corrected request. Unconfirmed details are stated.
Failure guarantee: No purchase or submission bypasses approval. Authentication follows FR23.

UC4: Correct or pause a running task

Primary actor: User
Preconditions: A task is active.
Trigger: The user says "stop working" or presses the stop key.
Main success scenario:

1. The user requests a pause while an approved event creation is in progress.
2. The voice interface recognizes the stop phrase and says "Pausing work" within STOP_ACK_TIME (FR3, NFR3).
3. The task manager enters Pausing. The gate blocks new task actions. Unused approvals are withdrawn. Safely cancellable work is interrupted.
4. The submitted event creation cannot be cancelled safely. It returns a matching event record. The system records that result (FR8, FR12).
5. The task enters Paused. The user hears "Paused. The event was already added. The remaining work has not run."
6. The user may ask questions while work stays paused. A clear request to continue resumes the task after state checks. Any remaining writes need fresh approval.

Extensions:

- **1a.** The user corrects the task instead. The agent receives the correction (FR6). Changed action details require fresh approval (FR9).
- **1b.** The user says only "stop." Speech stops and work pauses. The system asks what the user wants next (FR3).

- **4a.** A form was prepared but not submitted. Its context is saved where possible. The system says it was not submitted. Resumption requires a check that the page still matches the request.
- **4b.** The earlier submission times out. The task remains paused. The system reports its outcome as unknown. Reconciliation may continue without advancing the task. It does not retry blindly (FR11).
- **6a.** The user cancels. The task enters Cancelling and then Cancelled. Earlier actions are reported and are not undone automatically.

Success guarantee: No new task action starts after the recognized stop request until the user resumes. Checks may only reconcile earlier actions. The user hears what completed and what remains.

Failure guarantee: An uncertain external result stays labelled unknown. A confirmed pause is not presented as cancellation of an earlier submission.

UC5: Reconnect without duplicating an action

Primary actor: User

Preconditions: A task is active and the connection drops.

Trigger: The connection to the user is lost.

Main success scenario:

1. The task pauses (FR7). The client signals connection loss within CONNECTION_NOTICE_-TIME. A local sound is used if network speech is unavailable (NFR.Sa.2).
2. The user reconnects, or starts a new session later.
3. The system briefly reports the task's state: what completed, what was pending, and anything uncertain (FR18, FR21).
4. The user says "continue." The system checks relevant state before resuming. Any unused approval is requested again.

Extensions:

- **1a.** Approval was pending. It is withdrawn. On reconnection the user hears that the pending action was not executed. Resumption needs fresh approval (FR20).
- **1b.** An action was already submitted. Its outcome is checked before a retry. The same action ID never executes twice (FR11).
- **4a.** The user cancels instead of continuing. The task ends after cancellation handling. The user hears what had already completed.

Success guarantee: No completed action is repeated as a retry. The user hears the task state before it continues.

Failure guarantee: Uncertain state is reported. Work does not resume silently.

S.7 Prioritization

- Must (POC, by November 23, 2026): FR1 to FR3, FR8, FR9, NFR3.
- Must (before participant evaluation): FR4 to FR7, FR10 to FR16, FR18 to FR24, NFR1, NFR5, NFR.E.1 to NFR.E.4.
- Should (tune with usability evidence): conversational timing, FR17, NFR2, NFR4.

The POC shall demonstrate approval and work-stop controls for its selected email/calendar and browser workflows. This demonstrates feasibility for the tested action paths; it does not establish coverage of every integration or website action.

S.8 Verification and acceptance criteria

- Each functional requirement shall have at least one test using synthetic data and test accounts. Detailed procedures belong in the V&V Plan.
- Approval tests shall check writes and consequential commitments. No tested consequential action may start without matching approval. Tests shall also cover autonomous navigation and the FR24 exception.
- Pause tests shall cover a lookup and a submitted write. Record acknowledgment time and the time new actions are blocked. Verify that later results are reported accurately. A voice reconnection shall not resume work by itself.
- Recovery tests shall cover uncertain outcomes and stale approvals. A completed action shall not be repeated as a retry.
- Outcome tests shall use a meaningful connector response, a matching record, and a website confirmation. A queued request shall not be reported as delivered. Insufficient evidence shall produce an unknown result.
- Voice tests shall cover pauses within a sentence and short acknowledgments. They shall also cover corrections, interrupted explanations, and status requests. Record premature replies and missed interruptions. Use participant feedback to tune timing.
- Privacy checks shall verify that application history and logs exclude authentication codes. Provider processing and retention shall be reviewed separately.
- State properties P1 to P6 (S.9) shall be checked against recorded events. NFR2 shall be measured on a declared task suite. Report all attempts and failures.

S.9 Formalization: task state machine

Each task has one state $q \in Q$, where $Q = \{\text{Working, AwaitingClarification, AwaitingApproval, Executing, Pausing, Paused, Cancelling, Cancelled, Done, Failed}\}$. The partial transition function $\delta : Q \times \Sigma \to Q$ uses the events in Table 9. Figure 4 summarizes task control.

The running states are Working, AwaitingClarification, AwaitingApproval, and Executing. Pausing, Paused, and Cancelling are also active states. Cancelled, Done, and Failed are final. A session has at most one active task.

A work-stop request blocks new task actions as soon as it is recognized. Pausing records this change and requests safe interruption. It then enters Paused. An earlier external operation may still be pending. Its outcome is tracked separately. Reconciliation checks may continue while

paused. They may not advance the task. The report distinguishes unfinished work from unknown outcomes.

Cancellation uses Cancelling to block new actions and preserve earlier results. It then enters Cancelled. Earlier operations are not undone. An unresolved external submission does not justify repeating it.

Pending approvals are withdrawn on pause, disconnection, or cancellation. They do not expire merely because a connected user takes time to answer. Resumption requires a clear instruction and a check of relevant state. Unused writes need fresh approval. A new voice session alone does not resume work. New requests follow FR19.

Table 9: Task state transitions

From	**Event**	**To**
Working	required details missing or ambiguous	AwaitingClarification
AwaitingClarification	user answers	Working
Working	user corrects the task	Working
Working	consequential action or batch proposed	AwaitingApproval
AwaitingApproval	explicit approval of exact details	Executing
AwaitingApproval	user declines or changes details; pending action not run	Working
Executing	item outcome recorded; approved batch items remain	Executing
Executing	last approved item outcome recorded	Working
Working	all requested outcomes confirmed	Done
Working	no way to continue; remaining outcomes reported	Failed
any running state	work-stop request or bare “stop”; dispatch blocked	Pausing
any running state	disconnect or provider outage; dispatch blocked	Pausing
Pausing	context saved; safe interruption requested; outstanding operations recorded	Paused
Paused	status question, reconnection alone, or another stop request	Paused
Paused	clear resume instruction; relevant state checked; blocking uncertainty resolved	Working
any running state, Pausing, or Paused	explicit cancellation or confirmed task replacement	Cancelling
Cancelling	dispatch blocked; context and outstanding operations recorded	Cancelled

Required properties:

- P1: A write or consequential commitment starts only in Executing with matching approval. FR24 is the sole cookie-rejection exception.
- P2: Changed details invalidate approval. Pause, disconnection, cancellation, and task replacement withdraw all unused approvals.
- P3: A recognized work-stop request blocks new task actions until explicit resumption. Only checks of earlier outcomes may proceed. Pause feedback reports any external work still pending.

- P4: Final task states do not resume execution. Later evidence may update an earlier action's outcome in history.
- P5: Each consequential action has a recorded approval of its exact details before execution. Each approved batch item has a unique ID and executes at most once. FR24 is the stated exception.
- P6: A session has at most one active task. This includes Pausing and Cancelling.

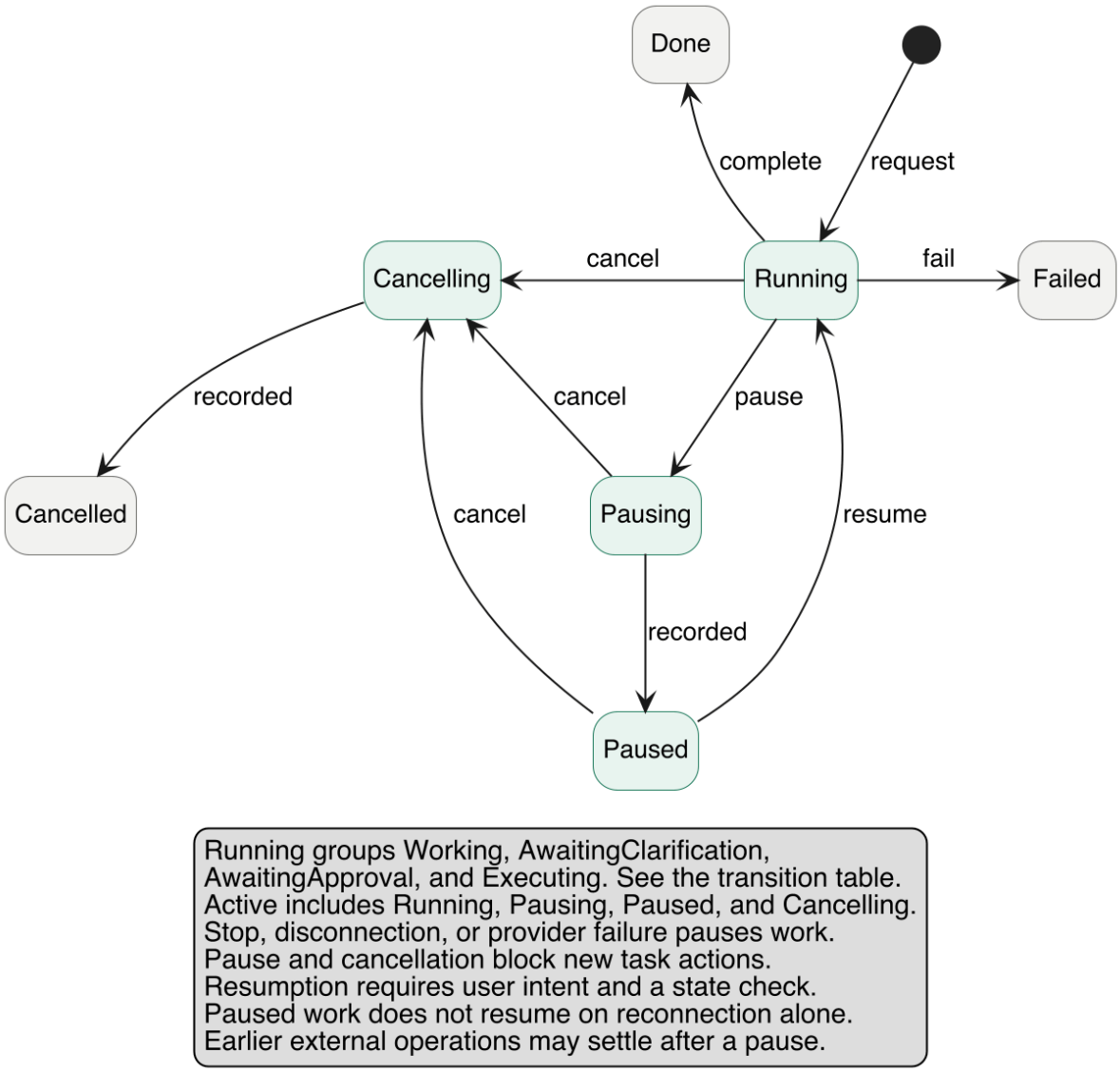


Figure 4: Task control states. Running groups the four running states in S.9. Grey states are final. External operations are tracked separately.

Project

P.1 Roles and personnel

As in the Development Plan: voice and client (Lucas, reviewed by Curtis); agent and tools (Aaron, reviewed by Kevin); deployment and reliability (Kevin, reviewed by Aaron); evaluation and quality (Curtis, reviewed by Lucas).

Comment: *Decide who writes which SRS sections.*

P.2 Imposed technical choices

- Hermes agent harness, LiveKit, Browser Use, Composio (for Gmail and Google Calendar).
- Mac, Chrome, English for the initial release.
- LaTeX documents in the course template repository; GitHub for issues and reviews.

P.3 Schedule and milestones

- October 19: SRS and Hazard Analysis.
- November 2: V&V Plan.
- November 16: Design document revision minus one; Interview Report.
- November 23 to December 3: POC demonstration.
- January 18: Design document.
- March 8: V&V Report and Usability Report.
- March 31: Final documentation. April 6: EXPO.

P.4 Tasks and deliverables

Comment: *Break the POC into tasks on the GitHub project board (owner, acceptance criteria, due date) and summarize them here in a table.*

P.5 Required technology elements

- LiveKit Agents (Python) and a speech-to-text and text-to-speech provider.
- Hermes agent and an AI model provider.
- Browser Use with Chromium.
- Gmail and Google Calendar APIs through the Composio SDK, our own Google OAuth app before participant sessions, and test accounts.
- TypeScript and React for the web client.

P.6 Risk and mitigation analysis

- Supervision through Hermes may not work as documented: test approvals, interrupts, and reconnection on a pinned version first.
- Browser actions may not be interceptable for approval: if not, the browser becomes read-only for the initial release.
- Recruitment of blind participants may fail: report technical feasibility only and disclose missing usability evidence.
- Provider costs may exceed the budget: meter usage and reconcile monthly.

P.7 Requirements process and report

Requirements come from the Problem Statement, Development Plan, Hazard Analysis, TA guidance, prior research, and interviews. An interview with a former DBCS staff member was held on October 17, 2026; FR16 to FR18, the G.8 profile change, and intervenors as stakeholders came from it. Blind participants will be recruited for repeated feedback. Requirements will be revised as POC results and interview findings arrive.

Requirement dependencies

Comment: *Build a traceability matrix: goals G1 to G5 against FR1 to FR24, NFRs, and FR.Sa requirements.*

Likely and unlikely changes

- Likely: replacing the AI model, speech provider, or agent harness; adding account services; changing timing constants after measurement.
- Unlikely: dropping the approval rule for consequential actions; dropping voice as the main interaction.

References

Comment: *List every source cited (WCAG, AODA, PIPEDA, Google policy, LiveKit, Hermes, Browser Use, WebAIM, Morae, Meyer handbook).*

Appendix: AI use disclosure

Aaron Loh used Claude Code (Claude Opus 5.5) to work through the requirements. It asked questions, summarized tool documentation, and translated the team's decisions and Kevin's requirements draft into this structured draft. Aaron made and reviewed every decision. The team will review, verify, and rewrite this draft before submission.

OpenAI Codex assisted with the October 17 layout and consistency review. It also helped revise the supervision requirements and simplify the System section from Aaron's decisions. Team review is pending.

Comment: *Other members: add the tools you used.*

Appendix: Reflection

Comment: *Each member answers the SRS reflection questions from their own experience.*

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Kevin Kim

1. **What went well while writing this deliverable?**

   One thing that went well was how our team divided responsibilities. Aaron and I focused on the System section, while other members worked on areas such as the goals and Hazard Analysis. This made the workload more manageable. Using AI tools to review our work also helped identify inconsistencies and missing requirements. I focused on making the requirements realistic and feasible, which helped improve the overall consistency of the SRS.

2. **What pain points did you experience, and how did you resolve them?**

   The main challenge was resolving inconsistencies between the SRS and Hazard Analysis. Some requirements described different system behaviors, so I compared both documents and helped determine which approaches were more feasible and appropriate. Another challenge was ensuring the requirements were specific enough to implement and test. Reviewing requirement traceability and refining overly broad requirements helped us address these issues.

3. **How many requirements were inspired by speaking to clients or their proxies?**

   I did not personally participate in stakeholder interviews while writing the SRS. Most of the requirements I worked on came from team discussions, where we considered the needs of blind users, along with our project goals and technical constraints. I cannot confidently identify an exact number of requirements directly inspired by stakeholder interviews. Moving forward, gathering feedback from blind users will help us validate our assumptions and improve our requirements.

4. **Which courses will help your team succeed?**

   Software Requirements, Human-Computer Interfaces, and Software Testing are three courses that I think will be especially useful. Software Requirements helped us develop and organize the SRS. Human-Computer Interfaces will help us design an accessible voice interface for blind users, while Software Testing will help us verify that the system meets its requirements, particularly for safety features such as approvals, interruptions, and error handling.

5. **What knowledge and skills will the team need to acquire?**

   Our team will need to improve our knowledge of AI agents, backend development, security, voice interfaces, and accessibility. Based on our Development Plan, I will focus on deployment, reliability, and secure backend systems. Aaron will need to develop his knowledge of AI agent frameworks and tool integration. Curtis will focus on accessibility testing and evaluation, while Lucas will need experience with voice interfaces and VoiceOver compatibility. We will also need to share knowledge to integrate these components successfully.

6. **How will each team member acquire these skills?**

   Two useful approaches are studying documentation and tutorials, and building practical prototypes. I prefer learning by doing, so I plan to experiment with backend services and error handling while using documentation and AI tools for guidance. Aaron can prototype AI agent integrations, Curtis can review accessibility guidelines and apply them through testing, and Lucas can build and test voice interactions with VoiceOver. Hands-on development will help us identify problems and learn from experience.

Aaron Loh

1. **What went well while writing this deliverable?**

   Working through the requirements really helped me and the team clarify what we are actually building. It really forced us to think through the actual use cases of how it might be used and the flows that users would go through. We were also able to work really well as a team on dividing different things, going through the important parts of our requirements together, and coming up with a shared vision for everything before we went our separate ways.

2. **What pain points did you experience and how did you resolve them?**

   One of the hardest things was defining our scope and really keeping it consistent across both the Hazard Analysis and the SRS. For example we really wanted to be ambitious with what we're doing but we also needed to keep it scoped so we definitely had some discussions there.

   Another part that was a little bit difficult was defining what stopping work means. We had to really think through what should happen in different situations to make that clear in our requirements.

3. **How many requirements were inspired by speaking to clients or their proxies?**

   I had the chance to interview a former CNIB Deafblind Community Services staff member. Their perspective on their experience and their answers to the various questions that I had were very helpful in understanding our users better, or at least a certain subset of our users. Soon we'll be planning to go through some of these things with actual blind users.

4. **Which courses will help your team succeed?**

   Software Engineering 3RA3 will definitely be a big one. Human-Computer Interfaces, which I'm taking this semester, will be really helpful as well in thinking about how people are going to be interacting and interfacing with this. It is very different from any traditional interface so we will certainly be thinking from first principles.

   Software Design will also be an important one. The things we learned in 3A04 will help because there's certainly going to be good design that needs to happen here to make sure our system works well and is easy to build on top of.

5. **What knowledge and skills will you need to acquire?**

   I think one of the biggest learnings and pieces of knowledge that I'll need to gain is in the agent ecosystem: specifically understanding the latest and greatest of how people are building these things but also how they're combining them together and designing these systems well.

   Understanding how our software can interact with the external world in a way that's flexible and truly agentic is definitely going to be an interesting topic to understand better. Designing that in a way that's secure and safe is also going to be something we're going to have to learn a lot about.

6. **How will you acquire this knowledge and these skills?**

   I would say it's going to be a lot of prototyping as well as reading through documentation, explaining what I've learned, and seeing how it applies to our own problem. Then by prototyping and really diagramming the systems I'll be able to understand them.